



Rooted

Technical Documentation

A Bible study companion for iOS and Android — offline Scripture, a prayer journal, study and sermon notes, Scripture quizzes, and community study plans.

Version	1.0.0 · iOS build 10
Bundle identifier	com.rootedbible.app
Platform	Expo SDK 54 · React Native 0.81
Repository	github.com/Sarbah12/ROOTED
Apple team	Accra Resource Center LBG (3FPZL5YV7Z)
Generated	07 August 2026

Contents

1. Overview and status
2. Architecture
3. Bible
4. Notes, prayer and quiz
5. Community study plans
6. Moderation
7. Authentication
8. Data model
9. API reference
10. Configuration
11. iOS builds and release
12. Design decisions
13. Known gotchas
14. Setup checklist

1. Overview and status

Rooted brings Bible reading, a prayer journal, study and sermon notes, Scripture quizzes and shared reading plans into one app. The complete King James Version ships inside the binary, so reading works with no network at all.

By the numbers

Area	Figure
Bundled Scripture	66 books · 1,189 chapters · 31,100 verses
Translations available	17 public domain, plus 3 licensed once configured
Quiz bank	75 questions across 16 books and 12 topics
App screens	17
Data hooks	10
Database tables	13
Backend route groups	17
Commits	25

Maturity

This section is deliberately blunt. A feature list that mixes *works* with *compiles* is how a project ends up surprised.

Verified by running

What	Evidence
TypeScript	tsc --noEmit passes with 0 errors
iOS build and upload	EAS build 10, signed, accepted by App Store Connect
Bible reader	John 3 rendered from the bundle in a running app
Bundled text integrity	Genesis, Psalms, Revelation, Malachi, 3 John, Obadiah checked
Sign-in identifier detection	Username, phone and email each classified correctly
Sermon note fields	Preacher fields appear only in sermon mode
Translation picker	8 language groups, NT-only versions badged

Built but never executed

None of the following has run against a live system, because the backend has no database or Firebase credentials yet. Expect adjustments on first contact.

Area	Blocked by
Password sign-in, sign-up, reset	Firebase Email/Password provider not enabled
Everything touching Postgres	No database provisioned; schema never applied

Area	Blocked by
Transactional email	No message sent; sender domain unverified
Study plans, streaks, reflections	Whole API surface untested
Notes and prayer sync	Offline half works; sync half has not reached a server
Sign in with Apple	Entitlement correct, flow never exercised

The honest summary

The app works offline today: reading, quizzes, notes, prayers and settings all persist on device. Everything that crosses the network is written, typechecked and reviewed, but unproven.

2. Architecture

A React Native client with an offline-first data layer, talking to a small Node API over Postgres. Authentication is delegated to Firebase; the database stores only application data, keyed by the Firebase user id.

Repository layout

Path	Contents
app/	Screens, file-based routing via expo-router
app/(tabs)/	Home, Bible, Notes, Prayer, Quiz
app/plans/	Study plan browse, create, detail and day screens
constants/	Theme, Bible metadata, quiz bank, identity mapping
assets/bible/kjv/	Bundled Scripture, one JSON file per book
hooks/	Data access: notes, prayers, plans, chapters, quiz
context/	Auth and settings providers
backend/	Node API, with its own package.json
scripts/build-bible.mjs	Regenerates the offline Bible bundle
Rooted/	Legacy snapshot. Ignore; excluded from tsconfig

Offline-first data layer

Notes and prayers share one implementation, `hooks/use-synced-collection.ts`. The device is the source of truth: every edit writes to `AsyncStorage` immediately and the interface never waits on a request. When the user is signed in and the backend is reachable, pending work is pushed and remote records pulled.

Records carry a pending flag of create, update or delete. Local edits win over server state, and deletions are held as tombstones until the server confirms. A journal entry must not vanish because a request failed.

Backend has its own dependency tree

`backend/package.json` is separate by necessity. The app keeps `firebase-admin` in `devDependencies`, so a production install of the backend from the app manifest would silently omit it.

3. Bible

The complete King James Version ships with the app: 66 books, 1,189 chapters, 31,100 verses, under `assets/bible/kjv/`. Reading needs no network.

Generated files — `constants/bible-books.ts` and `constants/bible-offline.ts` — must not be edited by hand. Regenerate with:

```
npm run build:bible
```

The loader map is lazy, so opening one book does not parse all 66.

Translations

Group	Versions
Bundled	KJV
English, full Bible	WEB, WEBBE, ASV, Douay-Rheims 1899, Darby, BBE
English, New Testament only	Young's Literal, Open English Bible (US and Commonwealth)
Other languages, full	Latin Vulgate, Portuguese (Almeida), Romanian, Czech
Other languages, NT only	Russian Synodal, Chinese Union, Cherokee
Licensed, pending a key	NKJV, NLT, AMP

Coverage is verified, not assumed

Each translation was checked by requesting Genesis 1. Six carry only the New Testament. They are badged accordingly, and requesting an Old Testament book falls back to the bundled KJV with an explanation rather than failing with a bare 'not found'. Note that Young's Literal shipped unlabelled before this check.

Licensing

Every bundled and freely fetched translation is public domain, so none needs a licence. This was a deliberate correction: the app originally shipped NIV-worded sample text, which is copyrighted by Biblica and would have required an agreement for App Store distribution.

NKJV, NLT and AMP are copyrighted and are licensed through API.Bible. The integration is built and inert until a key exists. The API key is a licensing credential and is proxied by the backend — embedded in the app bundle, anyone could extract it, and misuse bills to the account.

Plan	Cost	Terms
Starter	Free	Three copyrighted Bibles, 5,000 calls/month, strictly non-commercial
Pro	\$29+/month	Copyrighted Bibles, 150,000 calls/month

Ads, fees or upsells disqualify the free tier. NIV commercial use is not offered at any price.

4. Notes, prayer and quiz

Notes and sermon notes

A note is either a study note or a sermon. Selecting Sermon reveals preacher, church or event, series and the date preached; a plain study note stays plain. The list filters by All, Study or Sermons, and search reaches the preacher, church and series as well as the title and body — so a preacher's name finds every sermon of theirs.

Prayer journal

Requests are tracked by category with a status of unanswered, ongoing or answered, and can be tied to Scripture. Same offline-first machinery as notes.

Quiz

Questions carry both a book and one or more topics, so a single bank of 75 questions feeds two browsing modes: study John then quiz on John, or study the parables across the gospels then quiz on Parables. Questions shuffle per attempt; best score and attempts are stored per subject.

Mode	Subjects
By book	16 books with enough questions to be worthwhile
By topic	Creation, Faith, Prayer, Parables, Miracles, The Cross, The Law, Wisdom, Prophecy, The Early Church, Salvation, Spiritual Warfare

Real data only

No screen shows a fabricated figure. Counts come from the user's own records, plan progress is days completed over plan length as reported by the server, and streaks are computed from actual completion dates.

Zero, never a placeholder

When data cannot be loaded, screens show zero or an empty state. Cached values are kept on failure because they were real once; a guess is never substituted. Earlier revisions hardcoded reading plans at 23%, 58% and 12% complete and pre-highlighted John 3:16 as though the reader had marked it.

5. Community study plans

A user authors a plan with a passage for each day, then shares it. Members mark days complete, see one another's progress, and write what they learnt that day.

Visibility

Setting	Who can reach it
Private	The owner only
Link	Anyone holding the six-character join code
Public	Listed in the directory for anyone to find

Enforced in one place, `getPlan()`, rather than repeated at each route.

Progress and streaks

Consecutive days. A member's position is the furthest *unbroken* run of days finished. Someone who jumps to day 20 without doing 2 to 19 still shows as day 1; otherwise the member list would mislead.

Calendar days. A streak counts distinct dates across every plan and survives if the reader was there today or yesterday. Three plans in one day is one day of streak, and a late-night reading does not break it.

Screens

Screen	Purpose
Browse	My plans and a public directory, plus join by code
Create	Title, visibility, and a day-by-day passage list
Plan detail	Progress, day checklist, invite code, member positions
Day	The passage from the bundled KJV, the prompt, the reflection feed

Creating a plan offers auto-fill, which walks consecutive chapters from the first entry and stops at the real end of the book rather than inventing chapters that do not exist. Ticking a day updates immediately and rolls back if the request fails, so the interface never claims something saved when it did not.

6. Moderation

Reflections are visible to everyone on a plan, which makes Rooted a user-generated-content app. App Store Review Guideline 1.2 requires such apps to provide a way to report objectionable content and to block abusive users. Apps are rejected without it, so this is not optional.

Control	Behaviour
Report	Offensive, spam or off-topic, from a menu on every reflection
Auto-hide	A reflection hides once three distinct people report it
Block	The blocked user disappears from every feed, member list and the directory
Delete	Authors can remove their own reflections

Auto-hiding at three independent reports means obvious abuse disappears before a human reviews it, while a single disgruntled reader cannot silence anyone.

Still required before review

A published privacy policy URL and a support contact are also conditions of approval for a UGC app. Neither exists yet.

7. Authentication

Sign-in accepts an email, a username or a phone number, each with a password, or Sign in with Apple.

Firebase provides no phone-and-password or username-and-password provider, so `constants/identity.ts` maps the latter two onto synthetic addresses on domains we own:

```
ada@example.com → ada@example.com (used as-is)
adalovelace → adalovelace@users.rootedbible.app
+233 20 123 4567 → 233201234567@phone.rootedbible.app
```

A useful consequence: Firebase already enforces one account per email address, so usernames and phone numbers inherit that uniqueness with no separate collision check.

Deliberate choices

Choice	Reason
One vague failure message	A specific error would reveal whether an account exists. The reset screen reports success even for unknown addresses for the same reason.
Recovery email at sign-up	Requested when the identifier is a username or phone, since neither is a real inbox and a forgotten password would otherwise be unrecoverable.
Branded reset email	The backend asks Firebase Admin for the reset link and delivers it through Resend, rather than sending Firebase's own template from a firebaseapp.com sender.

Console step required

Firebase → Authentication → Sign-in method → enable Email/Password. Until then every sign-up fails with `auth/operation-not-allowed`.

8. Data model

Postgres, hosted on Supabase. `backend/schema.sql` is idempotent and doubles as the migration.

```
cd backend && npm install && npm run migrate
```

Table	Holds
users	Profile keyed by Firebase uid, plus username and recovery email
user_settings	Theme, reminders, reminder time, font size
notes	Study and sermon notes, with preacher, church, series, date
prayers	Requests with category, status and verse
reading_progress	Position within a reading plan
quiz_results	Best score and attempts per subject
study_plans	Authored plans with visibility and join code
study_plan_days	The passage and prompt for each day
plan_members	Membership and furthest consecutive day
plan_completions	One row per day finished, dated for streaks
plan_reflections	What a member learnt on a given day
content_reports	Moderation reports
user_blocks	Who has blocked whom

Per-user isolation was a fix, not a given

The original store kept one global notes and prayers array in a single JSON file, so every signed-in account read and wrote the same rows. Every query now filters on the Firebase uid.

Row Level Security

RLS is enabled on every table with no permissive policies. The backend connects as the postgres role and enforces access itself, so RLS is not what gates these tables — but anything reaching the database through Supabase's public anon key reads nothing.

9. API reference

All routes except health require a Firebase ID token in `Authorization: Bearer <token>`.

Authentication and profile

Method	Path	Notes
POST	/v1/auth/login	Firebase idToken, returns the profile
POST	/v1/auth/logout	Revokes refresh tokens
POST	/v1/auth/password-reset	Always answers 200 — see below
POST	/v1/auth/welcome	Welcome email
GET	/v1/me	Profile and settings
GET PUT PATCH	/v1/me/settings	
GET	/v1/me/streak	Current and longest

Content

Method	Path	Notes
GET POST	/v1/notes	
PATCH DELETE	/v1/notes/:id	
GET POST	/v1/prayers	
PATCH DELETE	/v1/prayers/:id	
GET POST	/v1/quiz/results	
GET PATCH	/v1/reading-plans	
GET	/v1/bible/versions	Licensed versions this key allows
GET	/v1/bible/:bibleId/:book/:chapter	Proxied licensed text

Study plans and moderation

Method	Path	Notes
GET POST	/v1/plans	?scope=mine public, ?code=ABC123
GET DELETE	/v1/plans/:id	Delete archives
POST	/v1/plans/:id/join · /leave	
GET	/v1/plans/:id/members	Every member's position
POST DELETE	/v1/plans/:id/days/:day/complete	

Method	Path	Notes
GET POST	/v1/plans/:id/days/:day/reflections	
PATCH DELETE	/v1/reflections/:id	
POST	/v1/reports	Report content
GET POST	/v1/blocks	List or add
DELETE	/v1/blocks/:userId	Unblock
GET	/v1/health	Reports database reachability

Why password reset always returns 200

Answering differently for a registered address would turn the endpoint into an account-existence oracle. The app treats any response other than 503 as success for the same reason.

10. Configuration

Copy `.env.example` to `.env`. Secrets never enter the repository; `.gitignore` covers `.env`, `*.p8`, `*.p12`, `*.mobileprovision` and `credentials.json`.

Variable	Purpose	Required
DATABASE_URL	Supabase connection string	Yes
FIREBASE_PROJECT_ID	Token verification	Yes
FIREBASE_CLIENT_EMAIL	From the service account JSON	Yes
FIREBASE_PRIVATE_KEY	From the service account JSON	Yes
EXPO_PUBLIC_BACKEND_API_BASE_URL	Where the app looks for the API	Yes
RESEND_API_KEY	Transactional email	For email
RESEND_FROM	Verified sender address	For email
API_BIBLE_KEY	Licensed translations	Optional
CORS_ORIGINS	Comma-separated allowlist	Optional

Without the Firebase values, every authenticated endpoint returns 503

Confirmed by calling `/v1/me` directly: 'Firebase config is missing'. This is the single largest blocker to anything server-side functioning.

Deployment

`backend/render.yaml` is a Render blueprint; every secret is marked `sync: false` and must be set in the dashboard. iOS blocks plaintext HTTP, so a device build needs an HTTPS backend URL in the production profile of `eas.json`.

11. iOS builds and release

```
npx eas build --platform ios --profile production
npx eas submit --platform ios --latest
```

Profile	Target	Credentials
development	Dev client, simulator	Required
preview	Simulator, unsigned	None needed
preview:device	Real device	Required
production	App Store and TestFlight	Required

Identifiers

Bundle identifier	com.rootedbible.app
Apple team	Accra Resource Center LBG (3FPZL5YV7Z)
App Store ID	6798345216
EAS project owner	accra-resource-center

com.rooted.app was already taken in Apple's global namespace, hence the change.

Version numbering

appVersionSource is remote, so EAS owns the build number and auto-increments it. Do not set `ios.buildNumber` in `app.json`; it is ignored and EAS warns about it.

Credential creation cannot be automated

EAS refuses to generate a distribution certificate in non-interactive mode, by design. The first build must be run by a person answering the prompts. Once certificates exist, or a local `credentials.json` points at them, later builds run unattended.

App Store readiness

Requirement	State
Bundle ID registered	Done
Sign in with Apple capability	Done
Encryption declaration	Done — <code>ITSApplUsesNonExemptEncryption</code> false
App icon and splash	Done, generated from the Rooted mark
Report and block controls	Done
Category	Not set — Reference and Education suggested
Content rights	Not set — answer 'no third-party content'
Age rating	Not set — answering none throughout gives 4+
Privacy policy URL	Not written — required

Requirement	State
macOS platform entry	Should be removed; this is an iOS app

12. Design decisions

The reasoning behind choices that are not obvious from the code.

Public-domain translations only

The original sample text was NIV-worded, which is copyrighted by Biblica. Every translation shipped or fetched is now public domain, so no licence is needed and App Store distribution carries no rights exposure.

Identifiers map onto synthetic emails

Firebase offers no phone-and-password or username-and-password provider. Mapping both onto addresses at domains we own gets uniqueness enforcement for free.

Consecutive-day progress

Counting the furthest unbroken run, rather than total days done, stops someone who skips ahead from appearing further along than they are.

Zero, never a placeholder

Any figure that cannot be loaded renders as zero or an empty state. Cached values are kept on failure because they were real once; a guess is never substituted.

Licensed Bible keys stay server-side

An API key in the app bundle can be extracted by anyone who unzips the archive, and misuse is billed to the account. Public-domain text still goes straight from the device, since there is nothing there to protect.

Moderation built alongside the feature

Shared reflections make this a UGC app under Guideline 1.2. Reporting and blocking were built with the feed rather than retrofitted after a rejection.

reCAPTCHA vendored, then deleted

expo-firebase-recaptcha was unmaintained and pulled in expo-firebase-core, which uses pre-SDK-44 autolinking that no longer works. It was vendored to remove the native dependency, then deleted entirely when phone OTP gave way to passwords.

13. Known gotchas

Keep the project out of iCloud

This one cost hours. ~/Desktop and ~/Documents are iCloud-synced, and syncing a node_modules tree makes ordinary operations unusable. Measured on the same machine on the same day:

Operation	In ~/Desktop (iCloud)	In ~/Developer
git clone	10 min, timed out	4 seconds
npm install	26 minutes	21 seconds
git status	6+ min, timed out	1 second
tsc --noEmit	killed six times	3 seconds
eas build	hung twice, never uploaded	succeeded

The project now lives in ~/Developer/Rooted.

tsconfig excludes the legacy snapshot

Rooted/ is a second, older React Native app with its own node_modules. Without the exclusion every typecheck pulls it in and reports around seventy irrelevant errors.

Storage keys are versioned

Note records changed shape when sermon fields were added, so the AsyncStorage key moved from rooted:notes:v1 to :v2. Bump the suffix whenever a stored shape changes.

Credential hygiene

Two live credentials — an App Store Connect key and a Resend API key — were pasted into a chat transcript during development and should be rotated. Neither is in the repository; the exposure was the transcript. Local signing material lives in ~/Developer/rooted-credentials/ with 0700 permissions, outside the project directory.

14. Setup checklist

In order. The first three block everything server-side; nothing that crosses the network functions until they are done.

#	Step	Where
1	Enable the Email/Password sign-in provider	Firebase → Authentication → Sign-in method
2	Download the service account JSON, set the three FIREBASE_ variables	Firebase → Project settings → Service accounts
3	Copy the connection string into DATABASE_URL, then run the migration	Supabase → Connect → Direct connection
4	Verify a sending domain, or use the sandbox sender	Resend → Domains
5	Deploy the backend and set the production URL in eas.json	Render, or any HTTPS host
6	Set Category, Content Rights and Age Rating	App Store Connect → App Information
7	Publish a privacy policy and add its URL	Required for a UGC app
8	Rotate the two exposed credentials	App Store Connect and Resend
9	Optionally add an API.Bible key for NKJV, NLT and AMP	scripture.api.bible

One command once configured

`cd backend && npm install && npm run migrate` — the schema is idempotent, so it is safe to re-run after every change to schema.sql.

Rooted — growing deep in the Word.